ARCH — BRAINSTORMING — questions(IMP)

Status	Not started

🧠 A. System Design Thinking

Core Architecture

- What architecture will I use?
  - Monolith (recommended here) vs microservices?
  - ANS: monolithic as:
    - scope is controlled and keep it simpler as domains like users, financial records etc are tightly coupled.
    - faster development as there wont be any service communication.
    - clarity, maintainability, development speed
- How will I structure layers?
  - Routes(API) → Services → Repository → DB - why?
    - routes layer/API layer
      - accept request, validate input, call service, return response
    - service layer
      - business logic lives
    - repository layer
      - talks to the DB and runs queries
    - DB layer
      - for storage
- Where does business logic live?

- (Hint: NOT in routes)
- ANS: no idea

API Design

- REST or GraphQL? (→ **Use REST**, simpler + expected)
- How will I version APIs? (`/api/v1/...`)
  - using URL based versioning to ensure backward compatibility
- What are my core endpoints

👤 Users

```
POST   /api/v1/users              → create user
GET    /api/v1/users              → list users
GET    /api/v1/users/{id}         → get user
PATCH  /api/v1/users/{id}         → update user
DELETE /api/v1/users/{id}         → deactivate user
```

🔐 Auth (optional but strong signal)

```
POST   /api/v1/auth/login
GET    /api/v1/auth/me
```

💰 Financial Records

```
POST   /api/v1/records
GET    /api/v1/records
GET    /api/v1/records/{id}
PATCH  /api/v1/records/{id}
DELETE /api/v1/records/{id}
```

With filters:

```
GET /api/v1/records?type=income&category=food&start_date
=...
```

📊 Dashboard

```
GET /api/v1/dashboard/summary
GET /api/v1/dashboard/trends
GET /api/v1/dashboard/category-breakdown
```

Data Flow Questions

- How does a request move through my system?
  - ANS:

    Client (Frontend/Postman)
    ↓
    API Route Layer (FastAPI endpoint)
    ↓
    Authentication Middleware (identify user)
    ↓
    Authorization Check (role-based access)
    ↓
    Service Layer (business logic)
    ↓
    Repository Layer (DB queries)
    ↓
    Database (PostgreSQL)
    ↓
    Response back up the chain
- Where do I validate input?
  - ANS: Input validation is enforced at the API layer using schema validation (e.g., Pydantic in FastAPI). While frontend validation can improve user

experience, backend validation is mandatory to ensure data integrity and prevent invalid or malicious input.

    - TYPES OF VALIDATION:
        - schema ⟶ FastAPI/Pydantic
        - business ⟶ service layer
- Where do I enforce permissions?
    - ANS:
        - middleware/dependency layer ⟶ is user logged in? extract role
        - service layer

🗄️ B. Database Design (PostgreSQL – good choice)

Schema Design Questions

- What tables do I need?
    - ANS:
        - users
        - roles
        - user_roles
        - financial_records

Optional:

- categories
- audit_logs

Relationships

- One user → one role OR many roles?
    - ANS: i think one user can have many roles as admin will have all role permissions

- Should roles be enum or separate table?
  - ANS: roles table - id, name

👉 Better answer:

- Use a **roles table** (more scalable)

Financial Records Questions

- Should "type" (income/expense) be enum?
  - ANS: controlled values, as values wont change
- How do I handle categories?
  - ANS: separate table
- Should I normalize categories or keep them as strings?
  - ANS: normalize categories - it gives better querying, avoids duplication, cleaner analytics

---

Performance Thinking

- What fields need indexing?
  - `user_id`, `date`, `category_id, type`
- Will dashboard queries be heavy?
  - ANS: query is heavy if it scans many rows, uses aggregation, runs frequently

---

🔐 C. Access Control (VERY IMPORTANT — scoring area)

From the assignment:

Ask yourself:

- Where will I enforce RBAC?
  - ANS: Middleware/dependency later(auth + role extraction)

        - verify user(JWT/session)
        - attach user + roles to request
    - service layer
        - business rules → if user → perform action
- How do I check: “Can this user perform this action?”

Role	Read	Create	Update	Delete
Viewer	✅	❌	❌	❌
Analyst	✅	❌	❌	❌
Admin	✅	✅	✅	✅

Design Decision

- Role-based OR permission-based?
    - ANS: role based - RBAC

👉 For this assignment:

- Use **RBAC (simpler, cleaner)**

Questions

- What happens if:
    - Analyst tries DELETE?
        - ANS: deny access, return 403 forbidden
    - Viewer tries POST?
        - Return 403
        - Log the attempt
- LOGGING is important for debugging and audit trail

FINAL DESIGN (What your system should look like)

```
Request
↓
```

Authentication (middleware)
↓
User + Roles attached
↓
Service Layer:
→ Check permissions
→ Apply business logic
↓
Repository
↓
DB

📊 D. Dashboard Logic (Where most people fail)

This is NOT CRUD — this is **aggregation design**

Questions:

- Do I compute on-the-fly OR cache?
  - on the fly initially, caching later
- How do I calculate:
  - Total income
  - Monthly expenses
  - net balance
  - category wise breakdown
  - monthly trends

💰 Total Income

```
SELECT SUM(amount)
FROM financial_records
WHERE type='income';
```

💸 Total Expenses

```
SELECT SUM(amount)
FROM financial_records
WHERE type='expense';
```

🧮 Net Balance

```
SELECT
    SUM(CASEWHEN type='income'THEN amountELSE0END)-
    SUM(CASEWHEN type='expense'THEN amountELSE0END)
FROM financial_records;
```

🔥 Better (cleaner query)

```
SELECT type, SUM(amount)
FROM financial_records
GROUPBY type;
```

👉 Then compute in backend

📊 Category-wise Breakdown

```
SELECT category_id, SUM(amount)
FROM financial_records
GROUPBY category_id;
```

📅 Monthly Trends

```
SELECT DATE_TRUNC('month',date)ASmonth,
       SUM(amount)
```

```
FROM financial_records
GROUPBYmonth
ORDERBYmonth;
```

Example queries:

```
SELECT SUM(amount)FROM recordsWHERE type='income';
```

Advanced Thinking (optional but impressive)

- Should I use:
  - Materialized views?
  - Precomputed tables?
- ANS: For this assignment, I use on-the-fly aggregation queries. For scalability, I would consider materialized views or precomputed summary tables to optimize frequently accessed dashboard metrics.

⚙️ E. Backend Framework Choice (Python)

Your Stack (Recommended)

- **Framework**: FastAPI ✅ (best for this assignment)
- ORM: SQLAlchemy or Prisma (Python version)
- DB: PostgreSQL
- Auth: JWT (optional but strong signal)

Questions

- Why FastAPI over Django?
  - ANS: I chose FastAPI because it provides a modern, lightweight framework with built-in support for asynchronous programming, automatic request validation using Pydantic, and auto-generated API documentation via Swagger. Compared to Django, FastAPI offers greater control over

architecture and is better suited for building clean, modular APIs for this assignment.

- Will I use sync or async DB calls?
  - ANS: I will use asynchronous request handling with FastAPI. For database interaction, I can start with synchronous ORM usage for simplicity, but the design remains compatible with async database drivers if scalability becomes a concern.

🧪 F. Validation & Errors

Questions:

- What happens if:
  - Amount is negative?
  - Missing fields?
- Do I use:
  - Pydantic schemas? (→ YES if FastAPI)
    - Pydantic is used for request validation and data parsing, ensuring type safety and automatic error handling at the API boundary.
- error handling
  - consistent format
  - meaningful messages
  - correct status codes

Case	Code
Missing fields	422
Unauthorized (no login)	401
Forbidden (no permission)	403
Not found	404
Server error	500

🧾 G. Documentation (High scoring area)

- Will I use Swagger? (FastAPI gives it free)
- Will I write a README explaining:

What your README MUST include

📌 1. Project Overview

- What system does

📌 2. Tech Stack

- FastAPI
- PostgreSQL
- SQLAlchemy

📌 3. Architecture

- layered design
- modular monolith

📌 4. Database Design

- tables + relationships

📌 5. Access Control

- RBAC explanation

📌 6. API Endpoints

- summary table

📌 7. Setup Instructions

```
pip install-r requirements.txt
uvicorn main:app--reload
```

📌 8. Assumptions

Example:

- each record belongs to one user
- roles are predefined

📌 9. Tradeoffs

Example:

- chose on-the-fly aggregation over caching

🧠 H. “Evaluator Mindset” Questions

From the doc, they care about:

- clarity
- maintainability
- structure

So ask:

- If someone reads my code in 5 min:
  - Can they understand it?